

Complete example

```
# LEGO slot:0 autostart
from gadget import *

async def main():
    robot = await Gadget.setup(
        left_motor=port.A,
        right_motor=port.B,
        wheel_diameter="small",
        turn_factor=1.447,
        color_sensor=port.C,
        distance_sensor=port.D,
        force_sensor=port.F,
    )

    robot.log("Drive forward/backward at a heading.")
    await robot.drive(distance=30, angle=0)
    await robot.drive(distance=-10, angle=0)

    robot.log("Turn to an absolute global heading.")
    await robot.turn(angle=90)
    await robot.turn(angle=0, long_turn=True)

    robot.log("Use exactly one stop condition per call.")
    await robot.drive_until(colors=[GREEN, WHITE], angle=-90)
    await robot.drive_until(sensor_distance=15, angle=0)
    await robot.drive_until(pressed=True)

    robot.log("Turn right (default) or left for colors.")
    await robot.turn_until(colors=[BLACK])
    await robot.turn_until(colors=[WHITE], left_turn=True)

    robot.log("completed")

runloop.run(main())
```

Remember: Movement functions are asynchronous and require `await`.

Gadget.setup()

```
await Gadget.setup(
    left_motor, right_motor,
    wheel_diameter, turn_factor,
    yaw_angle=0, yaw_face=motion_sensor.TOP,
    default_drive_velocity=320, default_turn_velocity=200,
    default_acceleration=1000, default_deceleration=1000,
    default_stop_type=motor.SMART_BRAKE, steering_correction=1,
    color_sensor=None, force_sensor=None,
    distance_sensor=None, drive_pair=motor_pair.PAIR_1,
)
```

Parameter	Purpose / accepted value
<code>left_motor</code>	Required. Left drive motor port. Must differ from right.
<code>right_motor</code>	Required. Right drive motor port. Must differ from left.
<code>wheel_diameter</code>	Required. "small" (5.6 cm), "large" (8.8 cm), or a positive diameter in cm.
<code>turn_factor</code>	Required. Positive yaw-to-motor multiplier calculated with <code>turn_factor.py</code> .
<code>yaw_angle</code>	Initial global heading, -180° to 180° . Default: 0.
<code>yaw_face</code>	Hub face pointing up. Default: <code>motion_sensor.TOP</code> .
<code>default_drive_velocity</code>	Driving speed, 100–1050 °/s. Default: 320.
<code>default_turn_velocity</code>	Turning speed, 100–1050 °/s. Default: 200.
<code>default_acceleration</code>	Acceleration, 100–10000 °/s ² . Default: 1000.
<code>default_deceleration</code>	Turn deceleration, 100–10000 °/s ² . Default: 1000.
<code>default_stop_type</code>	Motor stop behavior. Default: <code>motor.SMART_BRAKE</code> .
<code>steering_correction</code>	Nonnegative gyro correction. Default: 1.
<code>color_sensor</code>	Color sensor port. Default: None.
<code>force_sensor</code>	Force sensor port. Default: None.
<code>distance_sensor</code>	Distance sensor port. Default: None.
<code>drive_pair</code>	Motor-pair identifier. Default: <code>motor_pair.PAIR_1</code> .

drive()

```
await robot.drive(  
    distance, angle=None,  
    long_turn=False, velocity=None,  
    acceleration=None,  
)
```

Drives a distance in centimeters while using yaw to hold a global heading. The robot first turns when it is more than 1° from the requested heading.

Parameter	Description
distance	Centimeters to drive. Negative backs up; 0 only turns.
angle	Heading, −180° to 180°. Current heading if omitted.
long_turn	Use the longer initial route to <code>angle</code> .
velocity	100–1050 °/s; otherwise the drive default.
acceleration	100–10000 °/s ² ; otherwise the default.

Examples

```
await robot.drive(distance=30, angle=0)  
await robot.drive(distance=-15, angle=90)
```

drive_until()

```
await robot.drive_until(  
    colors=None, sensor_distance=None,  
    pressed=None, angle=None,  
    reverse=False, long_turn=False,  
    velocity=None, acceleration=None,  
)
```

Drives until **exactly one** sensor condition is met. Configure the matching sensor port in `Gadget.setup()`.

Parameter	Description
colors	Ordered color list. Copied; caller's list is unchanged.
sensor_distance	Stop at or below this sensor distance, 5–100 cm.
pressed	Use <code>True</code> to stop when force sensor is pressed.
angle	Heading to hold; current heading if omitted.
reverse	Drive backward when <code>True</code> .
long_turn	Use longer initial route to the heading.
velocity	100–1050 °/s; otherwise drive default.
acceleration	100–10000 °/s ² ; otherwise default.

Examples

```
await robot.drive_until(colors=[GREEN, WHITE])  
await robot.drive_until(sensor_distance=15, angle=0)  
await robot.drive_until(pressed=True)
```

turn()

```
await robot.turn(  
    angle, long_turn=False,  
    velocity=None, acceleration=None,  
    deceleration=None,  
)
```

Tank-turns to an absolute global heading. By default, the robot takes the shortest route.

Parameter	Description
angle	Target global heading, −180° to 180°.
long_turn	Take the longer route when <code>True</code> .
velocity	100–1050 °/s; otherwise turn default.
acceleration	100–10000 °/s ² ; otherwise default.
deceleration	100–10000 °/s ² ; otherwise default.

Examples

```
await robot.turn(angle=90)  
await robot.turn(angle=-90, velocity=250)  
await robot.turn(angle=0, long_turn=True)
```

turn_until()

```
await robot.turn_until(  
    colors, left_turn=False,  
    velocity=None, acceleration=None,  
)
```

Turns in place until the configured color sensor detects every color in the supplied order. The caller's list remains unchanged.

Parameter	Description
colors	Ordered list of colors to detect.
left_turn	Turn left when <code>True</code> ; right by default.
velocity	100–1050 °/s; otherwise drive default.
acceleration	100–10000 °/s ² ; otherwise default.

Examples

```
await robot.turn_until(colors=[BLACK])  
await robot.turn_until(colors=[GREEN, WHITE], left_turn=True, velocity=200)
```

Color values: BLACK, MAGENTA, PURPLE, BLUE, AZURE, TURQUOISE, GREEN, YELLOW, ORANGE, RED, WHITE, UNKNOWN.